



PDF Download
3604567.pdf
05 January 2026
Total Citations: 18
Total Downloads: 925

 Latest updates: <https://dl.acm.org/doi/10.1145/3604567>

RESEARCH-ARTICLE

Criticality-aware Monitoring and Orchestration for Containerized Industry 4.0 Environments

MARCO BARLETTA, University of Naples Federico II, Naples, NA, Italy

MARCELLO CINQUE, University of Naples Federico II, Naples, NA, Italy

LUIGI DE SIMONE, University of Naples Federico II, Naples, NA, Italy

RAFFAELE DELLA CORTE, University of Naples Federico II, Naples, NA, Italy

Open Access Support provided by:

University of Naples Federico II

Published: 10 January 2024

Online AM: 18 June 2023

Accepted: 04 June 2023

Revised: 16 March 2023

Received: 10 November 2022

[Citation in BibTeX format](#)

Criticality-aware Monitoring and Orchestration for Containerized Industry 4.0 Environments

MARCO BARLETTA, MARCELLO CINQUE, LUIGI DE SIMONE, and RAFFAELE DELLA CORTE, Università degli Studi di Napoli Federico II, Italy

The evolution of industrial environments makes the reconfigurability and flexibility key requirements to rapidly adapt to changeable market needs. Computing paradigms like Edge/Fog computing are able to provide the required flexibility and scalability while guaranteeing low latencies and response times. Orchestration systems play a key role in these environments, enforcing automatic management of resources and workloads' lifecycle, and drastically reducing the need for manual interventions. However, they do not currently meet industrial non-functional requirements, such as real-timeliness, determinism, reliability, and support for mixed-criticality workloads.

In this article, we present k4.0s, an orchestration system for Industry 4.0 (I4.0) environments, which enables the support for real-time and mixed-criticality workloads. We highlight through experiments the need for novel monitoring approaches and propose a workflow for selecting monitoring metrics, which depends on both workload requirements and hosting node guarantees. We introduce new abstractions for the components of a cluster in order to enable criticality-aware monitoring and orchestration of real-time industrial workloads. Finally, we design an orchestration system architecture that reflects the proposed model, introducing new components and prototyping a Kubernetes-based implementation, taking the first steps towards a fully I4.0-enabled orchestration system.

CCS Concepts: • **Computer systems organization** → **Real-time systems**; **Dependable and fault-tolerant systems and networks**; • **Software and its engineering** → **Real-time systems software**; **Monitors**;

Additional Key Words and Phrases: Real time, orchestration, mixed criticality, containers, Industry 4.0

ACM Reference format:

Marco Barletta, Marcello Cinque, Luigi De Simone, and Raffaele Della Corte. 2024. Criticality-aware Monitoring and Orchestration for Containerized Industry 4.0 Environments. *ACM Trans. Embedd. Comput. Syst.* 23, 1, Article 5 (January 2024), 28 pages.
<https://doi.org/10.1145/3604567>

This study was carried out in part within the MOST – Sustainable Mobility National Research Center and received funding from the European Union Next-GenerationEU (PIANO NAZIONALE DI RIPRESA E RESILIENZA (PNRR) – MISSIONE 4 COMPONENTE 2, INVESTIMENTO 1.4 – D.D. 1033 17/06/2022, CN00000023) and in part within the MICS (Made in Italy – Circular and Sustainable) Extended Partnership and received funding from the European Union Next-GenerationEU (PIANO NAZIONALE DI RIPRESA E RESILIENZA (PNRR) – MISSIONE 4 COMPONENTE 2, INVESTIMENTO 1.3 – D.D. 1551.11-10-2022, PE00000004). The work by L. De Simone was supported by European Union - FSE-REACT-EU, PON Research and Innovation 2014–2020 DM1062/2021 contract number 18-I-15350-6. This manuscript reflects only the authors' views and opinions, neither the European Union nor the European Commission can be considered responsible for them.

Authors' address: M. Barletta, M. Cinque, L. De Simone, and R. Della Corte, Università degli Studi di Napoli Federico II, Via Claudio 21, Naples, Italy, 80121; emails: {marco.barletta, macinque, luigi.desimone, raffaele.dellacorte2}@unina.it.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1539-9087/2024/01-ART5 \$15.00

<https://doi.org/10.1145/3604567>

1 INTRODUCTION

The vision of *Industry 4.0* (I4.0) foresees the automatic and continuous reconfiguration of factory floors and production lines to adapt to evolving business goals. This includes the automatic rearrangement of the production lines through collaborative robots, as well as the continuous reconfiguration, deployment, and monitoring of new industrial components [1, 2].

The manifold I4.0 use-case scenarios can include highly available distributed control systems [3], populated by virtualized Programmable Logic Controller (PLCs) [4], wireless connected heterogeneous *things* (i.e., the Industrial Internet of Things paradigm) [5], online predictive maintenance, digital twin-based production processes [6], and so forth. To support these use cases, more and more industrial environments leverage *Fog/Edge computing* paradigms to offload the high computational costs of, for example, Artificial Intelligence/Machine Learning algorithms from devices and robots to nearby located *micro datacenters*, better equipped in terms of resources [7, 8]. Furthermore, replacing traditional machinery with *softwarized* industrial elements, according to the *Software Defined Anything* trend, allows easier management, reconfiguration, and replacement of the components, avoiding frequent manual intervention.

However, the paradigm shift requires consolidating heterogeneous services on the same edge node, e.g., real-time control, machine learning algorithms, and high-speed data acquisition, each of them with different requirements [9]. Hence, the concepts of Fog/Edge computing and mixed-criticality systems become strictly intertwined, and the whole factory floor can be seen as a mixed-criticality system, with each node running jobs with different levels of criticality, i.e., the designation of the level of assurance against failure needed for a system component [10].

Virtualization is seen as an enabling technology for the implementation of mixed-criticality components since it provides support for resource abstraction, partitioning, and multiplexing, consolidating multiple Operating Systems (OSs) on the same hardware [11, 12]. However, critical components must often meet temporal requirements, and consolidating real-time and general-purpose systems on the same System-on-a-Chip (SoC) requires ad hoc solutions [13]. One drawback of software virtualization is the overhead that hinders the efficient usage of hardware resources. Therefore, over time several proposals have risen to mitigate the overhead, such as hardware-assisted virtualization or para-virtualization. In this perspective, lightweight virtualization technologies (e.g., containers) are gaining the limelight in industrial environments as a way to pack and deploy low-latency and bandwidth-efficient services with reduced overhead and higher scalability compared to classical hypervisor-based solutions [14–16]. Nevertheless, OS-level virtualization (i.e., containers) is known to suffer from a reduced degree of isolation compared to hypervisor-based virtualization; thus, it cannot be seen as a panacea [17]. Containers are also seamlessly integrated into *orchestration systems* (e.g., Docker Swarm [18], Kubernetes [19], Apache Mesos [20]), which foster fully automated management. Orchestration systems are distributed systems that provide the automatic placement, deployment, monitoring, and migration of the packaged software across the infrastructure, behaving as cloud operating systems that manage hardware resources while abstracting their use [21, 22]. They represent a valuable technology to manage *softwarized* industrial components and to allow the required flexibility and *intent-driven* reconfiguration.

Currently, available orchestration systems are designed and tailored to cloud requirements. Despite the efforts, support for hard real-time applications with different levels of criticality and nodes with uneven and limited computing resources, special devices, and different isolation guarantees is not mature yet [7, 23]. As a consequence, several studies investigated the feasibility of an orchestration-powered real-time cloud, integrating the support for real-time and high-availability services into Kubernetes [3, 24–28], the most widely adopted framework for managing containers, characterized by a rich feature set. Despite the existing solutions introducing the support for deploying real-time containers, criticality-aware orchestration is still at the dawn. Specifically,

current solutions lack determinism (e.g., time-bounded placement and migration of resources), support for heterogeneous platforms and devices (in terms of OS/hypervisor), and capability of addressing the different degrees of isolation required by real-time workloads [29, 30]. Furthermore, the heterogeneity of platforms, which feature Multiprocessor System-on-Chip (MPSoC), FPGAs, GPUs, and various I/O devices, leads to the need for solutions that integrate hybrid virtualization technologies, like hypervisor-based containers [31, 32].

To overcome current limitations, in this article we propose a *criticality-aware* orchestration solution for I4.0. In particular, we propose a model and an architecture enabling software placement, monitoring, and migration policies based on the criticality of the software component to be deployed. The objective is to guarantee the isolation of high-critical workloads while not impacting the performance of the less critical ones. The policies are based on the characteristics of nodes such as the isolation and timeliness provided by the hardware/software of the nodes, required network capabilities, and available resources.

In detail, our contributions are:

- Introducing a model for criticality-aware monitoring and orchestration of I4.0 workloads, taking into account (1) the real-time assurance to be provided by the resources of the node, (2) the criticality and real-time requirements of the workloads, and (3) the topology of the Fog/Edge infrastructure.
- Designing and prototyping an orchestration system architecture, namely *K4.0s*, which complies with the proposed model, by adding new components that handle the extended abstractions. Our proposal is based on Kubernetes and it is publicly available. The architecture is designed to be open to different virtualization approaches, like OS-level virtualization and hypervisor-based containers, in order to meet diversified requirements.
- Performing an experimental analysis to understand the behavior of real-time critical workloads running on top of variously configured nodes with co-located stress loads. Relying on the results, we propose a workflow to derive suitable metrics to build a monitor that depends on both workload requirements and node guarantees.

The article extends our previous work [33], where we introduced *K4.0s*, by (1) extending the previously proposed model with novel abstractions, e.g., introducing individual assurance levels for resources; (2) providing further experiments highlighting the need for novel monitoring strategies, based on both workload and node characteristics, including a monitoring overhead analysis and a schedulability comparison with a vanilla solution; and (3) proposing a workflow for selecting key monitoring metrics, generalizing the approach to any task-level metric.

The rest of the article is organized as follows. Section 2 discusses related work, while Section 3 provides some basic knowledge and terminology of orchestration systems used in the article. Section 4 schematizes the problem that prevents the use of orchestration in I4.0. Section 5 introduces our system model and the proposed abstractions, while Section 6 describes the proposed architecture and the details of our Kubernetes-based implementation. Section 7 presents our experimental campaign as well as the proposed process to determine the most useful monitoring metrics. Finally, Section 8 concludes the article.

2 RELATED WORK

Current orchestration systems, such as Docker Swarm [18], Kubernetes [19], and Apache Mesos [20], and monitoring solutions, such as Kubernetes Metric server [34], Prometheus [35], Telegraf [36], and Victoria metrics [37], are mainly designed for cloud environments, where the presence of both resourceful nodes and workloads that do not require strict timing constraints can be assumed. As such, they do not take into account I4.0 requirements, such as managing potential

heterogeneous environments, real-timeliness, and carrier-grade availability with reduced times to repair.

The usage of orchestration and lightweight virtualization technologies to build and manage applications with critical timing constraints represents a recent trend in the literature, which wants to take advantage of the scalability, flexibility, reconfigurability, increased reliability, reduction of costs, and consumption of resources. In this context, different solutions have been proposed for the orchestration of real-time containers. For example, *RT-Kubernetes* [24] is a modified version of the Kubernetes orchestrator, which leverages the *rt-cgroups* interface for the deployment of real-time containers that rely on the hierarchical modification to *SCHED_DEADLINE* described in [38] to schedule cgroups. An enhanced version of the Kubernetes scheduler and node agent, i.e., *kubelet*, makes use of an extended description of tasks to include real-time constraints. Similarly, the work in [26] modifies Kubernetes to enable the deployment of both real-time and best-effort containers. The Kubernetes control plane is modified to allow both admission control and scheduling of real-time containers, while an RT-manager is deployed on each working node for performance evaluation. Real-time containers are implemented in the same way as RT-Kubernetes, and node performance is evaluated by analyzing both container-level and system-level metrics during orchestration. A modified version of OpenStack is proposed in [39], where real-time containers have been integrated to foster the Network Functions Virtualization management for next-generation networks. In [40] Open-Stack is modified to integrate real-time Virtual Machines for Xen, using the compositional schedulability analysis. The work in [41] presents an architecture for real-time *Function as a Service (FaaS)*. The work aims to open cloud/edge computing infrastructures to time-critical applications, assuming functions running within containers. The proposed architecture formulates an analytical model considering the end-to-end latency of real-time request/response pairs, highlighting the importance of real-time communication. In addition, a partitioning algorithm is designed to minimize the number of used compute nodes. A Kubernetes-edge-scheduler is proposed in [25], which aims to enable reliability for edge applications through spare backup resources. The scheduler considers latency constraints of applications by extending the cluster scheduler with the awareness of a delay matrix. Similarly, in [3] container orchestration is seen as a chance for improving the recovery performance in a distributed control system scenario, complementing the manual replacement of failed controllers. A latency-aware placement is presented in [27], which describes a Kubernetes-based fog layer architecture for industrial automation contexts. The proposal is latency aware with regard to communication costs, aiming at reducing the amount of data transferred between sensing, computing, and actuating devices leveraging a greedy heuristic. Several extensions to Kubernetes are proposed in [28] in order to better deal with shared resources, such as cache and memory bandwidth, dividing the resources in discrete bands, and resizing pod resources for a collaborative Quality of Service (QoS) reduction. In [42] Kubernetes is modified to offer automatic and transparent state replication. To this aim, the proposal integrates both services coordination and a leader election protocol. In [43] is presented *RunPHI*, a framework that proposes to integrate mixed-criticality containers into orchestration systems. The framework leverages the strong isolation provided by partitioning hypervisors while taking advantage of orchestration techniques, paving the way to easy deployment of critical workloads in edge/cloud environments, failure mitigation through migration, and full reconfigurability for workloads with isolation requirements.

For completeness, we mention *K3s* and *K0s* as efforts to make Kubernetes more lightweight. *K3s* is stripped down to have fewer dependencies, cloud provider integrations, and add-ons, packing all control plane components in a single binary, with the ability to disable some components. *K0s* is similar to *K3s*, with an in-cluster database and storage support, and specialized workload support

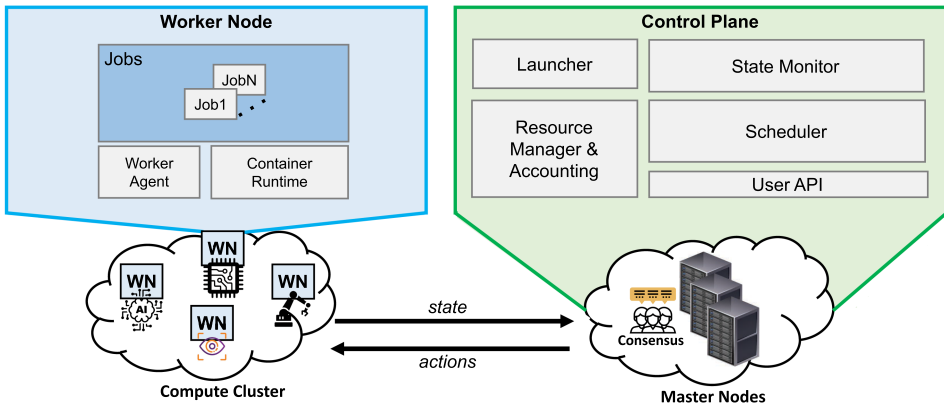


Fig. 1. Reference architecture of container-based cluster orchestration systems.

for serverless functions and IoT applications. Despite this, such efforts do not provide evidence for real-time guarantees and criticality awareness.

Past research trends also showed interest in deploying and managing real-time software components; an example is provided by real-time service-oriented architecture (SOA) [44–48]. For example, in [46] an architecture for real-time SOA in industrial environments is presented, highlighting the importance of real-time communication channels and resource reservation to meet specified QoS. In [45] is also highlighted the importance of real-time service discovery and composition, and real-time deployment, which requires a reserved communication bandwidth. However, these studies do not deal with the problem of how to select the node where to deploy a service, since they meant *orchestration* as the way the services were composed.

Compared to our closest works, we propose a model with finer granularity for real-time interfaces; in order to be extensible with several scheduling algorithms, we also take into account network scheduling, and we specialize the real-time capability of the node in multiple real-time assurance levels for the different resources, enabling criticality-aware monitoring and orchestration of I4.0 workloads. Hence, the main difference is in the model itself, which takes into account (1) the real-time assurance that can be provided by the resources of nodes, (2) the criticality and real-time requirements of workloads, and (3) the topology of the fog/edge infrastructure.

3 BACKGROUND

In this section, we provide background on container orchestration systems. Specifically, in Section 3.1 we present notions related to orchestration and a generic architecture common to multiple container orchestration systems (depicted in Figure 1), using a simplified version of the terminology introduced in [22]. In Section 3.2 we provide background concepts specific to Kubernetes, which is the orchestration system we used to implement the prototype of the proposed criticality-aware monitoring and orchestration architecture.

3.1 Container-based Orchestration

An orchestration system is a distributed system divided into two logical components: a *control plane* and a *compute cluster*. The former takes global decisions for the cluster, and it is in charge of receiving requests for deployments, monitoring the state of the compute cluster, and controlling the lifecycle of the *jobs*. A job is the minimum unit of scheduling for the orchestration system and consists of a set of related containers hosting one or multiple *tasks*; i.e., it is the smallest deployable unit of computing that can be created and managed. The containers belonging to a pod

are always deployed together, sharing resources like namespaces. An *application* is composed of one or more jobs. The control plane is deployed onto a *master node* (i.e., *cluster manager master* in [22]), or optionally on multiple consensual master nodes, to guarantee greater availability. The compute cluster is in charge of carrying out the jobs requested by the user, and it is composed of a set of *worker nodes*, on which jobs are spawned. The control plane issues commands to agents deployed on each worker node, while the agents periodically report the state of nodes to the control plane.

The **control plane** includes the following components: (1) a *Launcher*, which issues actions to the worker nodes; (2) a *State monitor*, which stores the global state of the worker nodes and jobs and issues actions when the state deviates from the desired one; (3) a *Resource manager and Accounting*, which is aware of resource availability within the compute cluster and tracks their usage for both scheduling and accounting purposes; (4) a *Scheduler*, which decides on which node to place a job, between the nodes that have enough available resources—the scheduler is also invoked when migration is triggered, i.e., in case of a failure of the hosting node or a preemption from a higher-priority job, or because the former hosting node does not match anymore the condition requested for the job; and (5) a *User API*, which receives deployment requests, validates them, and triggers the scheduler if needed. Each of these components communicates with the others to carry out its tasks. For example, the scheduler decides the node based on the free resources tracked by the resource manager and the state of the nodes tracked by the state monitor and then delegates to the launcher the task to issue actions.

The **compute cluster** is made up of several worker nodes, each of which runs (1) a *Worker agent*, which collects health information about the worker node itself and the jobs scheduled on the node; (2) a *Container runtime*, which is a container engine (e.g., Docker, LXC, etc.); and (3) the jobs deployed. The health information regarding the node is sent to the control plane, while the one about the jobs is used to restart them when necessary, communicating with the *container runtime* to keep the desired state over time and deploy jobs.

On a deployment request, the control plane decides the placement of the job on the worker nodes through a *scheduling* process. It generally relies on metrics related to resource consolidation in cloud computing, like free CPU and memory of the nodes, co-located jobs, and so forth, and assumes to have, unless properly instructed, nodes with similar computing and networking capabilities.

The control plane has also the duty to perform actions to keep the current state equal to the desired one given by the user. For example, an application can specify the desired number of load-balanced replicas of a service. If the control plane detects the failure of one of them, it reacts accordingly to restore the number of requested replicas. Failures are mainly detected through *heartbeat* mechanisms, at both the job and node level. A failure at the job level can be addressed by restarting the job on the same node. Conversely, a node failure must be addressed by spawning a replica on a different worker node, with a process hereon referred to as *migration*. A migration can also be triggered because the hosting worker node for a job does not respect anymore the placement constraints for the job; hence the job is spawned elsewhere, and the resources formerly allotted to it are released. For example, the jobs must be respawned elsewhere if the hosting node is not healthy anymore or it falls short of resources like free memory and disk.

3.2 Kubernetes

Currently, the most popular container orchestration systems are *Kubernetes*, *Docker Swarm*, and *Apache Mesos*. By extension, similar functionalities are provided by other cloud computing platforms such as *OpenStack*, which can handle both virtual machines and containers.

Between them, Kubernetes is the orchestration system with the biggest market share. It is highly configurable and extensible, and it offers many features [49]. Components in Kubernetes are loosely coupled, and they can be replaced by custom ones that expose the same interface. In Kubernetes terminology, a job is implemented through a *Pod*, which is a group of one or more containers. Some resources like networking are allotted to the whole Pod, while containers can specify requests and limits for CPU and memory, which are mapped to underlying configurations of the *cgroups*. Requests represent the minimum amount of resources that must be guaranteed to a container, while limits represent the maximum achievable usage before being throttled. A *Deployment* can specify the number of Pod replicas that must be alive: Kubernetes performs actions to restore the number of running replicas when they are less than desired. Since each Pod is characterized by an IP address, to easily reach and load-balance the replicas, Kubernetes provides the *Service* abstraction, thanks to which it masks all the replicas behind a unique IP. A working node in Kubernetes is abstracted with a *Node*, which is characterized by a state, some metadata like *Annotations*, *Labels*, *Taints* (conditions that prevent the scheduling or execution of some Pods on the node), and available resources in terms of CPU and memory. Custom resources can be defined for abstracting accelerators and external devices, which cannot be shared between Pods. The scheduler in Kubernetes is implemented through a dedicated Pod that waits for new scheduling events. The scheduling framework is composed of sequential phases (*filtering*, *scoring*, *reserve*, *permit*, and *bind*), and during each phase, the enabled *plugins* contribute to the scheduling decision. Custom plugins can be integrated, and a custom scheduler can be used instead of the default one. Moreover, Kubernetes offers features like the *high-availability* configuration, in which multiple master nodes elect a leader replica and update the state of the cluster through the *Raft* consensus algorithm, improving the system availability.

4 PROBLEM STATEMENT

Current orchestration systems are designed with cloud requirements in mind, and they are not suited to manage workloads with strict timing requirements over heterogeneous nodes with uneven and limited computing resources, accelerators, and devices. Hence, there is an inherent lack of abstractions that prevents the use of orchestration systems in I4.0 environments.

To meet industrial requirements, orchestration systems must be equipped with policies of placement, monitoring, and migration that take into account the real-time requirements and the criticality level of workloads, along with the isolation and timeliness (i.e., assurance) provided by the hardware/software of the nodes, networking capabilities, communication latencies, battery life, and so forth. In particular, the number and types of metrics to monitor both depend on the *resources* needed by the workload and on the capabilities of worker nodes to *ensure* the isolation of such resources in the presence of disturbances.

In the remaining part of the article, we identify and address the following three problems, which affect the existing container orchestration systems:

- **A system model, providing proper abstractions for the components in I4.0.** The abstraction of worker nodes, jobs, and networks must be redefined to take into account I4.0 needs. For example, abstracting a worker node as a couple of CPU and memory overlooks essential details. Even if there are ways to specify non-standard resources like GPUs, sensors, and so forth, worker nodes may drastically differ in terms of timing/security guarantees and execution times of the workloads, depending on the hardware/software configuration (e.g., use of real-time OSs and/or hardware features to avoid resource contention). Indeed, nodes in I4.0 range from embedded boards compliant with safety standards to edge servers. Similarly, nodes can also be interconnected by multiple networks of different types, each of

them with its own features and degree of isolation, such as Time-sensitive Networks (TSNs), Controller Area Network (CAN) buses, wireless *slices*, and so on. Therefore, the placement policies should adapt to workload requirements, considering an extended set of abstractions for both nodes and jobs.

- **An architecture for monitoring and orchestration in I4.0.** The components of both the *control plane* and *computing cluster* must be designed properly to fit the new system model and meet functional and non-functional requirements. For example, current orchestration systems are designed to be scalable, and they also allow the definition of suitable SLAs, but they do not provide criticality-aware placement policies that care about the impact of resource interference on application timeliness and predictability.
- **A framework for monitoring metrics selection in I4.0.** The strict non-functional industrial requirements ask for a monitoring different from the state of practice, based on an extended set of metrics, in order to timely reveal issues, such as loss of isolation, and apply proper countermeasures. The set of metrics on which the monitoring system must be built may strongly differ, depending on the requirements and characterization of each workload, along with the isolation degree for each resource provided by the node on which the workload is placed.

To substantiate the problems depicted above, we report in the following the results of an experiment performed in our previous work [33]. We used *rt-app*¹ as workload to generate synthetic real-time tasks, while collecting both activation latency and slack time metrics (more details are provided in Section 7). During the experiments the *containerized rt-app* is run for 30 minutes to collect, over time, the thread *activation latency*, defined as the difference between the actual and the expected start time, and the slack time between the deadline and the finishing time (*slack time* hereinafter) defined as $d - f$, where d is the task deadline and f is the finishing time. *rt-app* runs as a single-threaded application executing a periodic CPU-bound activity.

A stress workload is generated by running *stress-ng*² at the host level, through 16 concurrent stressors that generate different loads (the x-axis labels in Figure 2). In order to mitigate transitory situations and biases, we run the experiment 5 times, setting a maximum load to 16 concurrent stressors. The experiment is executed focusing on the following configurations for a worker node: (1) Linux with *rt-cgroups* [50] enabled (general-purpose kernel), (2) Linux with *rt-cgroups* enabled and *low-latency* preemption mode, (3) Linux with *rt-cgroups* enabled and *PREEMPT_RT* patch, and (4) Xenomai-patched Linux. We set the Linux kernel version to v5.15, and for Xenomai we used v3.1, which has support for real-time containers [51]. Besides the first configuration, power optimizations and frequency scaling are disabled.

The *rt-app* thread has a *period* of 10 ms and a *runtime* of the CPU consumption activity of 1.9 ms. The *runtime* defines the amount of time during which the application runs CPU-bound operations, according to a previous calibration. This *period* has been chosen to have a high frequency of samples, while the *runtime* of 1.9 ms prevents the budget limitation enforced by the different scheduling algorithms of the considered systems (the budget of Xenomai containers is set to 2 ms).

Figure 2 shows the obtained results. Whiskers are set to minimum/maximum of sampled values, while the box covers the interquartile range. Clearly, the general-purpose kernel cannot provide any guarantee, while the *low-latency* configuration shows bounded times in some cases, but there are still troublesome stress loads that can be hazardous for a real-time critical task. Note that we consider a worker node as *hazardous* if it negatively influences task execution in terms of key real-time application metrics like activation latency and slack time. In general, we adhere to the

¹<https://github.com/scheduler-tools/rt-app>.

²<https://kernel.ubuntu.com/~cking/stress-ng/>

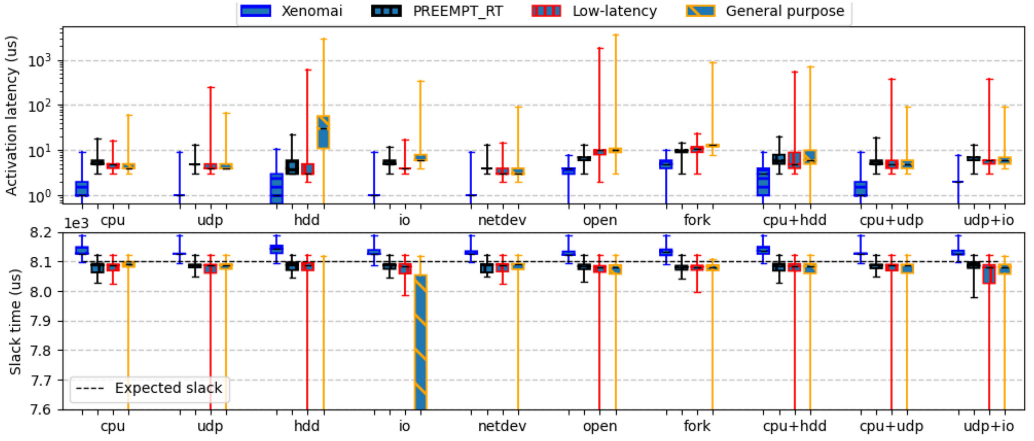


Fig. 2. CPU-bound workload behavior, with thread activation latencies (lower is better) and slack times (higher is better) under stress conditions (16 concurrent stressors) across different configurations. Slack times are clamped for readability.

dependability literature, which includes the so-called *timing failures* [52], for which the timing of service delivery (e.g., task execution time) deviates from implementing the system function (i.e., deadline miss).

PREEMPT_RT presents bounded errors, while Xenomai is the most predictable system, with negligible or null interquartile ranges. In general, *hdd* and *open* are the most troublesome stress loads for all targets, while the effect of the others depends on the specific target.

The results are not intended to make a comparison between the several solutions for containers, but to highlight that different worker node configurations provide varied isolation guarantees, which motivates the need for a new model of worker nodes encompassing *assurance levels* for real-time tasks. These assurance levels must be weighted according to the needs of a task, which can be more interested in respecting timings on certain events rather than others. The assurance levels must drive the selection of the monitoring metrics and the placement decision for containers.

5 SYSTEM MODEL

In this section, we present a system model that includes novel abstractions for the worker nodes, jobs, and networks.

Our target environment is a fog/edge cloud infrastructure with industrial requirements (e.g., a factory floor), since the I4.0 vision foresees fog/edge computing as a key paradigm to achieve flexibility and reconfigurability, respecting non-functional requirements like low response times. The environment encompasses a cluster of computing resources, made up of heterogeneous *worker nodes*, equipped with industrial devices, interconnected with one or multiple types of *networks*. As described in Section 3, a set of *master nodes* collects the *jobs* submitted by the users and assigns them to the worker nodes. By job we mean, differently from the usual meaning in real-time systems, the minimum unit of scheduling for the orchestration system, which consists of a set of related containers hosting one or multiple *tasks*.

We extend the abstraction of jobs with real-time requirements, criticality level, and requests for each shared resource. Therefore, the task includes a Worst-Case Execution Time (WCET) that is a function of both the worker node and the criticality level. We also extend the abstraction of worker nodes with real-time assurance levels for each of the shared resources. In the following paragraphs,

Table 1. Parameters of the System Model

Symbol	Meaning
$i \in [0, I[$	index of node
$WN_i = (\vec{BR}_i, AR_i, \vec{A}_i)$	worker node i
$\vec{BR}_i = [CPU_i, Disk_i, Mem_i]$	basic resources of node i
$AR_i = \{\vec{S}_i, \vec{ACT}_i, GPU_i, FPGA_i\}$	additional resources of node i
$\vec{S}_i, \vec{ACT}_i$	accessible sensors and actuators
$A_{i,r} = \alpha_{i,r} + \beta_{i,r} + \gamma_{i,r}(t)$	assurance level of node i for the resource r
$q \in [0, Q[$	index of network
$NW_q = (V_q, E_q)$	network q
$M_q : V_q^2 \rightarrow (E_q^d, E_q^c)$	delay and capacity matrix of network q
$j \in [0, J^{TOT}[$	index of job
$J^j = (\vec{BR}^j, AR^j, NW^j, C^j, RT^j, AP^j)$	job j
$\vec{BR}^j, AR^j$	basic and additional resources used by job j
$NW^j = \{NW_0^j, \dots, NW_{Q-1}^j\}$	network requirements of job j
$RT^j \in \{RT, non - RT\}$	real-time requirement of job j
$C^j \in \{NO, LOW, HI\}$	criticality of job j
AP^j	aggregation policy of job j
$k \in [0, K^j[$	index of tasks of job j
$TK_k^j = (WCET_k^j(i), T_k^j, P_k^j)$	task k of job j
$WCET_k^j(i)$	WCET of task k , job j on node i
T_k^j, P_k^j	period and priority of task k

we formalize the abstractions for each entity of the orchestration system. Table 1 summarizes all the symbols we used in describing the abstractions.

■ **Worker nodes and resources.** We assume a cluster composed of I heterogeneous worker nodes. Each worker node $WN_i = (\vec{BR}_i, AR_i, \vec{A}_i)$ $i \in [0, I[$ is a tuple composed of a vector of *basic hardware resources*, a set of *additional heterogeneous resources*, and a vector of *real-time assurance level* for resources, respectively. $\vec{BR}_i = [CPU_i, Disk_i, Mem_i]$ is the vector of basic resources, i.e., CPU, disk, and memory. $AR_i = \{\vec{S}_i, \vec{ACT}_i, GPU_i, FPGA_i\}$ is the set of additional resources that includes $\vec{S}_i$ and $\vec{ACT}_i$, which are respectively vectors of sensors and actuators that can be accessed from the node, as well as accelerators and/or hardware features, e.g., GPU and FPGA. $\vec{A}_i$ is the *assurance level* of WN_i . It is conceived as a vector of assurance levels of the individual resources: e.g., $\vec{A}_i = [A_{i,CPU}, A_{i,Disk}, A_{i,Mem}]$ is the assurance level of basic resources.

$\vec{BR}$ includes values representing the absolute available quantity of the resource. For the CPU, we use *CPU utilization* as the unit of measurement, defined as $U = C/T$, where C is the computation time and T is the period. Depending on the scheduling algorithms used, *CPU utilization* could be divided by *number of CPU cores*. AR_i is made up of integer values that represent how many resources are available in the node. We assume an exclusive allocation of a resource to *job* for the sake of simplicity. If a resource can be shared among up to N jobs, it can be modeled as N separate resources, taking into account the impact on the isolation assurance. The vector $\vec{A}_i$ also includes the additional resources, depending on their availability. Each element $A_{i,r}$ of the vector represents the degree of isolation that WN_i can provide for the resource r , based on its hardware/software characteristics. Hence, each $A_{i,r}$ can be decomposed into $A_{i,r} = \alpha_{i,r} + \beta_{i,r} + \gamma_{i,r}(t)$, where $\alpha_{i,r}$

and $\beta_{i,r}$ are pure numbers representing the assurance level scores associated to the availability of hardware and software assurance mechanisms, respectively. The parameter $\gamma_{i,r}(t)$ is a time function that represents the assurance level given by the state of the system at time t , that is, how current co-located loads may affect the isolation of the real-time critical task, as seen in the previous section. Hence, $\gamma_{i,r}(t)$ allows to temporarily reduce the assurance of a resource when a co-located anomalous workload turns out to potentially undermine the guarantees of a more critical job, and thus to represent a hazardous condition.

■ **Network topology.** The worker nodes are connected by one or multiple networks NW_q , $q \in [0, Q]$, with different purposes, requirements, and features. For example, we can consider a *best-effort network* connected with the Internet, a separate monitoring network, and other networks with real-time guarantees, such as TSN, CAN bus, or a *slice* of a wireless network.

NW_q is a graph described by (V_q, E_q) . $V_q = \{WN_0, \dots, WN_{|V_q|-1}\}$ is the set of nodes belonging to the network, while E_q represents the links between nodes. Links are characterized by a communication delay E_q^d and a capacity E_q^c , defining, in general, a matrix $M_q : V_q^2 \rightarrow (E_q^d, E_q^c)$. The orchestration systems should be aware of the topology of each network, because of their different features and set of nodes connected. For example, a job could need a sensor attached to a specific CAN network, and thus it is not sufficient to place it on a node with CAN capabilities; instead, the node topology is of interest. Furthermore, in a network with real-time guarantees, a sound schedulability analysis is needed to provide timeliness, as occurs in Time Division Multiple Access (TDMA) networks, common in both wired and wireless real-time networking standards [53]. Hence, the orchestration system must be aware of the network scheduling algorithms.

■ **Jobs.** Let J^{TOT} be the total amount of jobs to be scheduled in the target environment; a job J^j , with $j \in [0, J^{TOT}]$, consists of a set of containers with different running tasks (defined ahead) into them. A job is the *minimum unit of scheduling for the orchestrator*, and it has a set of tasks (see in the following) and allotted resources. Since the resources are allotted to the whole job, we do not care about how tasks and resources are split among its component containers.

$J^j = (\vec{BR}^j, AR^j, NW^j, C^j, RT^j, AP^j)$ is the tuple that describes the job J^j . $\vec{BR}^j$ are the basic resources required to carry out the job activities. AR^j are the additional resources the job can (optionally) take advantage of. Both of them have the same structure as the node resources. A job can also have a set of communication requirements $NW^j = \{NW_0^j, \dots, NW_{Q-1}^j\}$ on one or more networks. A job can optionally have real-time requirements ($RT^j \in \{RT, non-RT\}$) on resources. Considering the resource requirements, while the memory and disk needed by a job can be obtained by summing up resources needed by its tasks, CPU requirements need a more detailed analysis to stick to the scheduling theory, later discussed. Expressing the network requirements is strictly related to the network type, e.g., a slot for TDMA networks or a maximum round-trip time in a best-effort network. Special care needs to be taken for the placement of real-time jobs, as worker nodes have to be selected based on the real-time assurance of the resources required by the job.

We further introduce the notion of *job criticality*: different replicas of a job can be deployed, each of them having a different criticality level $C^j \in \{NO, LOW, HI\}$, e.g., a *leader replica* with a higher criticality level, and other backup replicas with lower criticality, potentially implementing Triple Modular Redundancy (TMR). Each criticality level should be backed by an appropriate assurance level of the worker node: a high-criticality job should be deployed on a node with a high assurance level $A_{i,r}$ on each of the resources requested by the job, i.e., implemented through a virtualization layer that provides a high degree of isolation, e.g., hypervisor-based containers with cache coloring, and so forth. Less critical jobs can instead take advantage of dynamic environments, with fewer guarantees. How the job criticality is backed by the assurance of the different resources must

be specified in an aggregation policy AP^j : a job can require a *minimum* level of assurance on each of the used resources, or it can require a minimum aggregated assurance score, given by the sum of the assurance of each resource weighted by the parameters specified in the AP by the job ($A_j = w_0 * A_{j,1} + \dots + w_r * A_{j,r} \geq \text{Threshold}$).

■ **Tasks.** A job is made up of a set of containers, each containing different tasks. K^j is the number of tasks composing the job J^j , and a task is defined as TK_k^j with $j \in [0, J^{TOT}[$, $k \in [0, K^j[$. In the case of a job with real-time requirements, its real-time tasks are described by an additional *real-time scheduling interface*. Assuming that the task requirements for both $\vec{BR}$ and AR are summed up and seen as requests for the whole job, we model a task as a tuple $TK_k^j = (WCET_k^j(i), T_k^j, P_k^j)$, where T is the period and P the (optional) task priority. Priority is optional since it is not considered in the task models of all scheduling algorithms. Unlike previous studies [24, 26, 40], we believe that the real-time scheduling interface of a job cannot be described by a model as simple as the (*period, budget*) pair: the budget needed by a container heavily depends on the WCET, which changes drastically upon the hardware, especially in a heterogeneous environment such as I4.0. Moreover, depending on the scheduling algorithm used, the CPU bandwidth needed by the whole job is usually more than the total sum of the CPU bandwidth needed by the single task. To overcome these limitations, in the proposed model each task TK is described by a period T and a WCET that is a function of the node and criticality level of the job ($WCET_k^j(i)$). Similarly to the approach of the Vestal model [54], the execution times depend on the criticality level: for high-criticality jobs, the WCET should be precisely computed for a set of specific hardware platforms, and only these platforms should be taken into account during the scheduling phase. Since this is not affordable for every application, a low-criticality job could leverage a cheaper method to provide a WCET estimation, at the cost of reduced confidence: for example, expressing the WCET as a baseline times a characteristic factor of the platform, requiring much less effort. Finally, a non-critical job, like soft real-time tasks, can be characterized by a cheap, more optimistic, and imprecise WCET estimation. Defining the precise WCET computation methods is beyond the scope of this work, and we generally assume a WCET that is a function of both the criticality level and the worker node considered.

Assuming a scheduling interface for each task, we are completely independent of how the scheduling framework joins task parameters to obtain a CPU request for the whole job. In this way, we cover the use of a plethora of *schedulability tests* on the nodes, with the only requirement that the tasks fit into the real-time scheduling interface, and the master nodes are not even aware of the specific scheduling algorithm used on the worker nodes.

6 MONITORING AND ORCHESTRATION ARCHITECTURE

In order to support the proposed system model, we present a monitoring and orchestration architecture that enriches the standard architecture of a container-based orchestration system as specified by Rodriguez and Buyya [22], by integrating new building blocks. These belong to both the *control plane* and *compute cluster* (see Section 3), because the control plane must enforce *criticality-aware* placement policies aware of the node resources, while the worker nodes must monitor each resource accordingly to the level of isolation guaranteed and notify the control plane about the assurance of the resources. Figure 3 shows the key modifications we present.

6.1 Control Plane

In the following, we describe the components to integrate into the Control Plane to adhere to the system model in Section 5.

► **Criticality-aware scheduler.** The *Scheduler* block is extended to support different placement policies for each *criticality level*. This is necessary since classical placement policies rely on

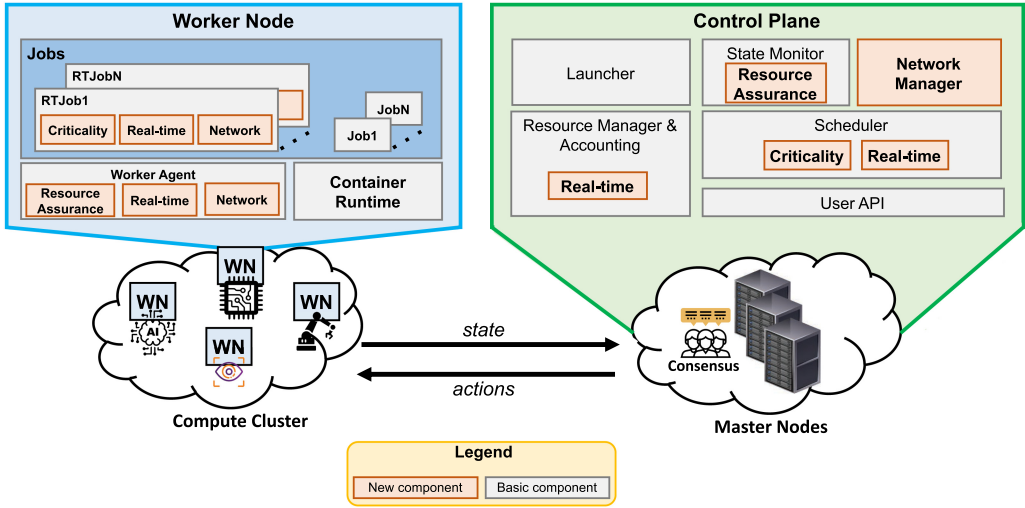


Fig. 3. Proposed architecture of k4.0s.

heuristics, but the proper strategy should depend on the specific criticality of a job. For example, for non-critical jobs (i.e., $\{Job1, \dots, JobN\}$ in Figure 3) consolidation may be more important than isolation, achieving better resource utilization and lower energy consumption. Conversely, critical jobs (i.e., $\{RTJob1, \dots, RTJobN\}$ in Figure 3) must be scheduled on nodes with high assurance, even at the cost of wasting resources. Moreover, the criticality level is linked with a *preemption level*, which allows critical jobs to preempt less critical ones in case of scarcity of resources.

► **Real-time support, control side.** The *Scheduler* and *Resource manager and Accounting* blocks need to support real-timeliness to meet stringent temporal constraints. The scheduler detects the *real-time scheduling interface* of jobs and forwards requests to a dedicated *scheduling agent*, which computes the *schedulability test*. This agent can be placed either on the master nodes or on each worker node. In the former case, the agent must be aware of every scheduling algorithm and current state of the nodes, as if it was integrated into the scheduler itself. In the latter, it must be aware only of schedulers and the state of its node. However, intermediate solutions are possible, like, for example, one scheduling agent for every specified number of nodes, with different impacts on the overhead of the scheduling process. We adopt the strategy of installing a *scheduling agent* for each node (the real-time module in the agent) to foster system flexibility: as long as a node provides an agent respecting the protocol, every scheduling scheme and every hypervisor/OS is allowed. Indeed, the orchestration scheduler is not aware of the particular schedulability test performed on worker nodes, and schedulability tests are *pluggable*. This enables the use of a broad range of virtualization approaches that host containerized applications, each characterized by different real-time frameworks.

► **Resource assurance monitor, control side.** This module collects information about the assurance levels of the worker nodes and their real-time capabilities over time. The *Scheduler* uses this information to implement a placement aware of the isolation assurance for the resources on the nodes, enforcing the aggregation policies to match the resources' assurance with the criticality of the jobs. For example, nodes with high assurance for specific resources are first selected for high-criticality jobs, while nodes with lower overall assurance levels are preferred for low-criticality jobs; this avoids the occupancy of highly isolated assurance nodes, which can be exploited for future critical tasks.

► **Network manager.** This component manages virtual networks and keeps track of the physical network topology and the different types of networks. Although this component already exists in several container orchestration systems, we highlight its importance and extend its duties. The scheduler delegates to it the schedulability test strictly related to the network requirements, both on the node and on the end-to-end communication path. For example, the *Network manager* keeps track of timeslots used in a TDMA-based network, delay matrices for best-effort networks, and available capacity for each physical link. The network manager coordinates and communicates with the network component of the *worker agent* to provide services usually offered in orchestration systems currently available, such as load balancers and proxies, which allow coping with rules of routing in complex virtualized networks. Similarly, we extend the collaboration between these components to provide routing services for critical applications. In particular, we argue that the network manager both on the control plane side and on the compute cluster side should enable *replica managers*, which keep track of the master and backup replicas of jobs and forward requests to the proper one, similarly to [42], and a *TMR voter*, which compares job responses and decides on majority.

6.2 Compute Cluster

We extend the basic Compute Cluster described in Section 3 with the components listed below.

► **Resource assurance monitor, worker side.** This component is running in the worker agent to extend its monitoring capabilities. In particular, the resource assurance monitor samples node-level metrics, which allow for obtaining the level of assurance for each resource of the worker node, and whether the worker node is appropriate to handle real-time and critical jobs. The worker agent monitor should take advantage of task-level metrics gathered by the job itself for more accurate analysis, tailored to the specific job, e.g., in the case of critical jobs. However, the assumption of having task-level metrics cannot hold for every job, and node-level metrics should be used to infer the node state.

The tuning of monitoring parameters, like sampling period, thresholds, and so forth, should be done according to the isolation guarantees provided by the underlying system. For example, if an anomaly is detected according to the state of a node, the sampling period could be reduced to better track the state evolution. Conversely, if a resource is well isolated, the monitoring overhead can be reduced, and the sampling period relaxed.

The resource assurance monitor operates in three stages. First, anomalies are detected through periodic sampling and trend estimation of key metrics, raising alarms if the worker node state is deemed hazardous for a specific real-time job. The alarm triggers the second stage, which tries to identify the guilty and enforce proper countermeasures. If the guilty is a low-critical job, it can be preempted or killed. In other cases, if after a timeout the monitor still judges the node as hazardous, it downgrades the assurance level of the involved resources of the worker node and notifies the control plane, which can decide to trigger a migration of the critical job.

► **Real-time support, worker side.** In the worker agent, this module accepts requests for real-time jobs using a predefined interface and protocol. The module computes the schedulability test for the incoming requests based on the resource availability and reserves such resources for the job. How to enforce the schedulability test and reserve resources for real-time jobs is strictly dependent on real-time schedulers supported by the target virtualization technology. This enables pluggable schedulers and a plethora of virtualization technologies, as long as the module is compliant with the interface and protocol.

► **Network module.** This module, similar to the *Network manager*, is already present within the worker agent. However, we extend its capabilities and duties, like the support for other classes of networks, such as TSN, TDMA, and so on. It also communicates with the *Network manager* to

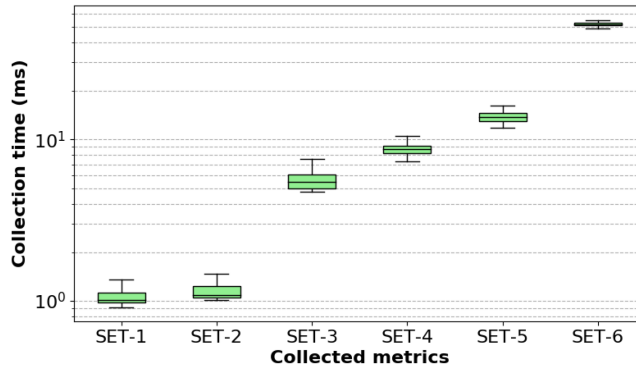


Fig. 4. Metrics collection overhead.

provide the fault-tolerance services described earlier. Finally, the module is responsible for allotting requested network resources to jobs.

► **Job abstraction.** Finally, the abstraction for a job is extended with the concepts of the model described before, i.e., criticality, real-time, and network requirements. Thus, a request for a job deployment can contain (1) the criticality level of the job, along with an aggregation policy that specifies the possible requirements in terms of assurance levels of single resources needed by the job; (2) the real-time scheduling interface for each task, including (in the case of a high-critical job) the WCET for the sole worker nodes where it has been computed; and (3) the networking and topology requirements (e.g., the length and the period for TDMA timeslots, or specific accesses to networks that connect requested nodes).

6.3 Key Metrics for Resource Assurance Monitor

The *resource assurance monitor* is a fundamental component, as it collects information about the assurance levels of the worker nodes and their real-time capabilities over time to properly drive the *criticality-aware* scheduler for the placement of jobs. Since many metrics could be available, potentially incurring a non-negligible monitoring overhead, there is the need to select the ones that are significant for a given node configuration. As an example, Figure 4 shows the collection time (in log scale) obtained by gathering various sets of monitoring metrics. The considered sets encompass different amounts of node-level metrics, which can be obtained through the `/proc` special filesystem of any Linux-based OS. Further, we consider sets including an increasing number of metrics, ranging from a baseline set, i.e., SET-1, which encompasses system load and uptime, CPU usage, and software interrupts statistics, to a more comprehensive set, i.e., SET-6, which includes more than 70 metrics that cover different system behaviors (including SET-1), such as *cgroups*-related metrics, memory, disk, network, node pressure, processes, and devices statistics. The remaining sets (i.e., SET-2-SET-5) represent intermediate levels, providing a subset of the metrics considered in SET-6, with each set including the previous one. As expected, the greater the number of collected metrics, the greater the collection overhead. Therefore, collecting a fully featured set of metrics for monitoring purposes can lead to non-negligible performance penalties. Further, note that the collection time is only a fraction of the monitoring overhead. Metrics are usually collected over time (sampling step), transformed in the desired format (parsing step), and analyzed (analysis step) for detection purposes. Despite the collection time appearing to be acceptable even for SET-6, it could be undesirable in the context of monitoring for real-time applications. Given that, this analysis highlights the need for properly selecting the most significant metrics to monitor worker nodes.

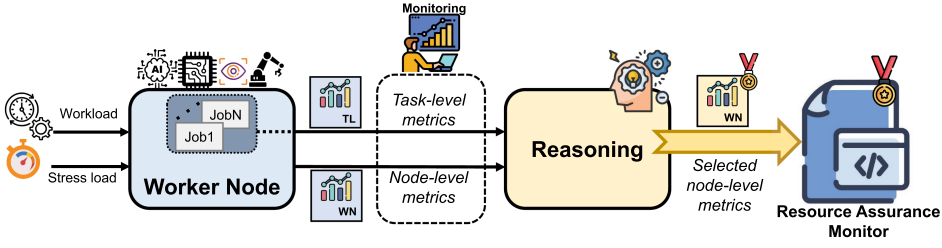


Fig. 5. Metrics selection workflow and resource assurance monitor generation.

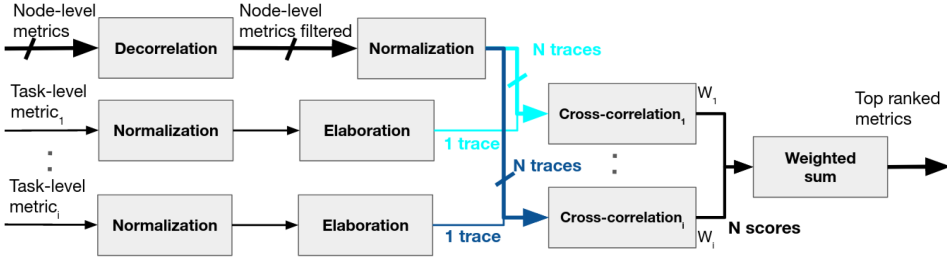


Fig. 6. Block diagram used for the reasoning process to extrapolate key node-level metrics.

To this aim, we designed a *reasoning process* to select the metrics, as depicted in Figure 5. A *workload* includes containerized real-time applications that perform CPU, memory, and I/O operations. We collect logs produced by the workload running on a worker node configured with different isolation capabilities (e.g., real-time support for kernel and network) and running a variety of co-located *stress loads*. The workload logs contain *task-level metrics*, which are linked to the timing of the different activities performed by the running tasks (e.g., task activation). These are fundamental to reveal potential issues regarding how the different resources are used. However, task-level metrics rely on the instrumentation of the application; hence their semantics is strictly dependent on the application, and they are not always available. Thus, to be agnostic to that metric, during the execution of the workload we also collected *node-level metrics*, for example, CPU usage, interrupts per second, and so forth. The proposed *reasoning process* correlates to both task-level and node-level metrics to understand the key node-level metrics for each combination of workload and worker node configuration on which an adequate *resource assurance monitor* must be based. This latter can be ultimately run on the worker node to communicate with the container-based orchestration system, regardless of the availability of task-level metrics.

According to the workflow presented in Figure 5, we implement the *reasoning process* by using the block diagram in Figure 6, which shows how we rank key node-level metrics for each pair (*workload*, *worker node configuration*).

We first filtered out highly correlated node-level metrics, e.g., metrics counting the same IRQ on different CPU cores, to avoid redundant metrics. Subsequently, the filtered node-level metrics are normalized to have the same importance independently of the absolute value. Different types of normalization are possible, e.g., average and standard deviation normalization or sum of squares normalization. At the same time, we consider i task-level metrics, each of them related to the timings of the different activities performed by the task within the target workload. Each task-level metric must be normalized in the same range and then passed to an elaboration block to give greater importance to sample spikes and outliers. This is necessary since we must consider

that the worst cases are paramount in a real-time scenario. Our implementation of the elaboration block is composed of a *max filter*, which replaces each sample with the maximum value within a sliding window of a determined number of samples, and an *exponentiation block*, which amplifies the spikes. For each node-level metric, we compute a score, obtained as the weighted sum of the cross-correlations with each task-level metric. The scores are used to rank the node-level metrics by relevance. The metrics with the highest score can be used by the *resource assurance monitor* to trigger alarms and detect the most troublesome workloads. The weights of the sum change based on the requirements of the job/task; hence they must be specified in advance with the deployment request to a worker node. Regarding the example of PLC (see above), the task-level metrics related to networking could be weighted greater than the others.

6.4 Criticality-aware Placement

We implemented the *criticality-aware* placement by extending Kubernetes, since it is a de facto standard in the context of containerized application orchestration due to the great extensibility out of the box. The source code of the prototype is publicly available at [55]. Since flexibility and extensibility are the key advantages of our architecture, the prototype demonstrates the feasibility of implementing specific extensions for real-time and network scheduling, i.e., the support for generic TDMA networks and hierarchical deferrable server schedulers (in particular, for the Xenomai implementation). Furthermore, the prototype includes the implementation of the selection of suitable monitoring metrics and criticality-aware placement.

For the control plane, we provide three *scheduler plugins* that implement the criticality-aware scheduler, real-time scheduling support, and a TDMA-enabling network manager. The criticality-aware scheduler filters out worker nodes that do not satisfy assurance requests and aggregation policies. Afterward, leveraging the aggregation policy, it scores them in decreasing/increasing order, looking for the assurance parameters in the Node *annotations*.³ The real-time scheduler plugin looks in the *Pod* annotations for real-time requirements, forwards them into a request for the *scheduling agent* running on worker nodes, and filters suitable nodes based on the response of the agent. The response contains the utilization for each core along with the schedulability result and allotted bandwidth if applicable. Finally, the scheduler scores the worker nodes using the available utilization for the least loaded core. The TDMA-enabling network manager relies on Pod annotations for TDMA network requirements, looks for a suitable timeslot, and then forwards the slot parameters to a TDMA master to set resources up.

The prototype includes three Kubernetes daemons to be deployed on worker nodes named *Malacoda*, *Rubicante*, and *Scarmiglione*.⁴ *Malacoda* is the assurance monitor, but at the moment it implements only a random modification of the node assurance level stored in the node annotations. *Rubicante* implements the real-time module within the worker agent: it receives scheduling requests, computes the schedulability test for a hierarchical deferrable server algorithm, and allocates required resources. Finally, *Scarmiglione* implements a TDMA master logic: it receives timeslot parameters and creates the slot on a Xenomai RTnet-based network, based on our previous work. Both the deferrable server and RTnet were used in our previous work to implement real-time containers in Xenomai [51]. Our design choice is to not modify the Kubernetes core for two reasons: (1) it allows us to replace the fully featured Kubernetes with other Kubernetes-compliant distributions, such as K3s or K0s [56, 57], and (2) it allows for a reduced maintenance effort in keeping pace with the Kubernetes releases.

³<https://kubernetes.io/docs/concepts/overview/working-with-objects/annotations/>.

⁴The names *Malacoda*, *Rubicante*, and *Scarmiglione* have been inspired by “Divina Commedia,” Dante Alighieri, 1321.

7 EXPERIMENTAL RESULTS

We present three sets of experiments with the aim to (1) highlight the need for the proposed model and architecture by analyzing the behavior of mixed workloads on different configurations of worker nodes, (2) show how the proposed assurance level can drive the selection of proper monitoring metrics, and (3) compare the scheduling capabilities provided by the proposal with respect to the vanilla version of Kubernetes.

The first two sets of experiments, namely *Mixed Workload Behavior* and *Node-level Metrics Selection*, are based on the workflow depicted in Figure 5. For the experimentation, we use the task model introduced in Section 5.

Experimental setup. Our experimental setup encompasses a four-core CPU Intel Core i7-4790, 16 GB DDR4 RAM, and 1 TB WDC WD10EZEX-60M HDD. It is important to note that all considered configurations have power optimizations (including C-states and P-states) and frequency scaling disabled at both kernel and hardware levels, with the frequency fixed at 3.6 GHz. Hyper-threading is disabled in all configurations. Three configurations are used for our worker node: (1) Linux with `rt-cgroups` enabled and `PREEMPT_RT` patch, (2) Xenomai-patched Linux with standard Linux networking stack, and (3) Xenomai-patched Linux with `RTnet` enabled [58]. All Linux kernels are set to the same version (i.e., v5.15), and for Xenomai we used v3.1. *RTnet* is a real-time networking stack shipped with Xenomai, which works with a pluggable real-time Medium Access Control (MAC) policy on standard Ethernet hardware. It offers, by default, a MAC TDMA policy [59], which consists in allocating a time slot to each physical node of the network for sending/receiving traffic.

7.1 Mixed Workload Behavior

In this section, we investigate the behavior of real-time containers under stress when they run a mixed workload (cpu-, memory-, and network-bound operations) on different worker node configurations. We focus on the ability of the worker node to isolate shared resources like CPU cache, network, and memory.

Experiment description. As workload, we run *rt-app*, a tool for creating synthetic real-time tasks by running one or more (optionally) periodic threads with a pre-defined behavior in terms of I/O activity, memory accesses, CPU operations, and so on. *rt-app* takes as input the description of the workload to emulate, in terms of the number of threads, scheduling details, period, and thread activity. Each thread produces logs in a pre-allocated memory area, including the measurements of activation latency and slack time.

We run the *rt-app* as a single-threaded application with a period of 10 ms and an execution loop that sequentially (1) writes 4,096 bytes to the main memory, (2) runs CPU-bound operations for 1.2 ms, (3) sends 1,024 bytes over the network interface, and (4) writes 8,192 bytes to the main memory. We set this loop to emulate a sequence of memory accesses, computation, and sending of the results, which is representative of edge computing services. The *rt-app* has been modified to allow network activity instead of standard I/O. We split the experiment into 11 phases, each of them with 80 seconds of cool-down and 60 seconds of running co-located stress. We applied the following stressors of *stress-ng*: *cpu*, *udp*, *hdd*, *io*, *netdev*, *open*, *fork*, *memcpy*, *cpu + hdd*, *cpu + udp*, *udp + io*. The last three options combine stresses belonging to the three categories I/O, network, and CPU to detect possible amplification or attenuation effects.

We collect the thread activation latency, the slack time, and the timestamp of sent packets through *tcpdump*. This latter is used to measure the inter-sending time between packets to reveal potential jitters that alter the periodicity of network packets, expected to be coherent with

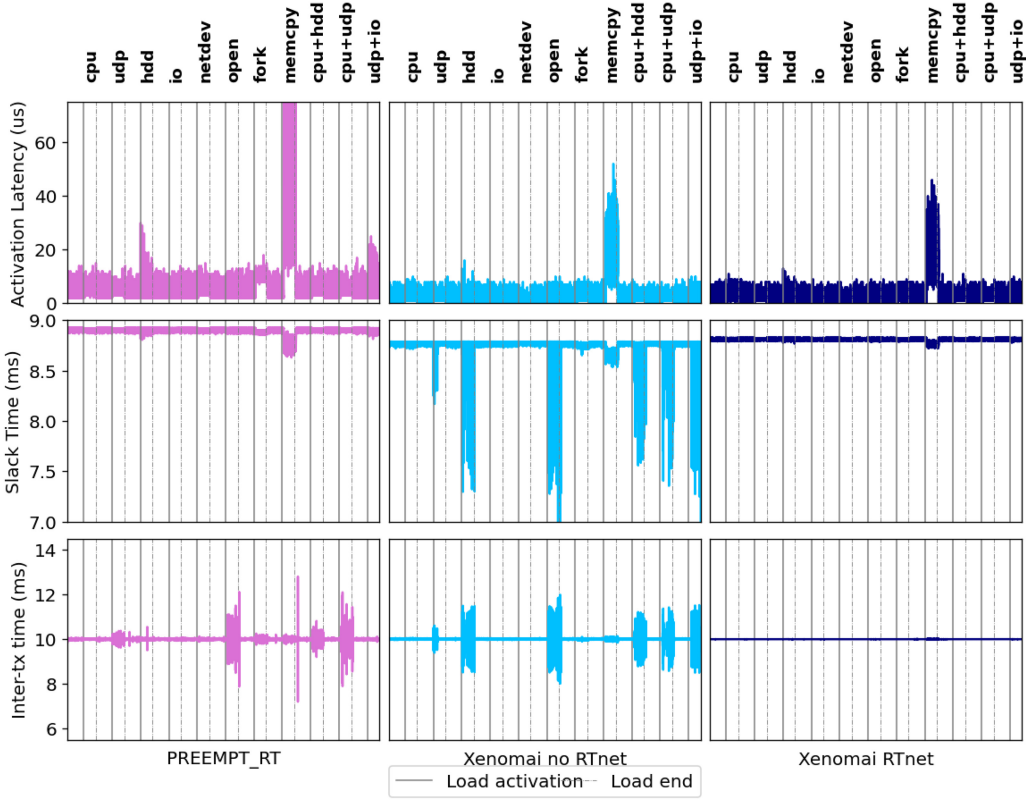


Fig. 7. Mixed workload behavior (thread activation latencies, slack times, and inter-sending times for network packets) by varying co-located stressors across target worker node configurations.

the thread period. We run the experiment for the three configurations presented in Section 7. During our experiment, the TDMA cycle is set at 5 ms, with 1 ms time slots. The TDMA master is the same machine on which we run experiments. Advanced isolation techniques of Linux, like the `isolcpus` and `nohz_full` kernel command-line parameters,⁵ along with IRQ redirection, are not used since they suffer from flexibility and are not integrated into container runtimes.

The experiment is repeated with an increasing number of stressor threads: 1, 2, 4, 8, and 16.

Results. Figure 7 shows the activation latency, slack time, and inter-sending time for network packets across the three target worker node configurations only in the worst case, i.e., 16 concurrent stressor threads. The results show a non-negligible interference on the shared resources (CPU cache, network, memory). We notice that *hdd* stressor has still an evident impact on the slack time and latencies against all the worker node configurations. At the same time, the *memcpy* stressor is clearly the worst across all worker node configurations. This behavior is expected since neither PREEMPT_RT nor Xenomai can benefit from cache isolation mechanisms like *cache coloring* [60] without relying on hardware features. Hence, the *memcpy* stressor evicts cache lines of *rt-app*, causing both high activation latencies and reduced slack times. However, the Xenomai configuration suffers slightly less than the PREEMPT_RT one if we consider the activation latencies. This is due to the way Xenomai schedules Linux tasks along with the Xenomai real-time tasks.

⁵<https://www.kernel.org/doc/html/latest/admin-guide/kernel-parameters.html>.

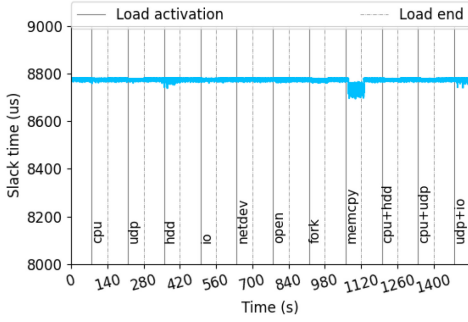


Fig. 8. Mixed workload slack time, no network traffic, in *Xenomai no RTnet* configuration.

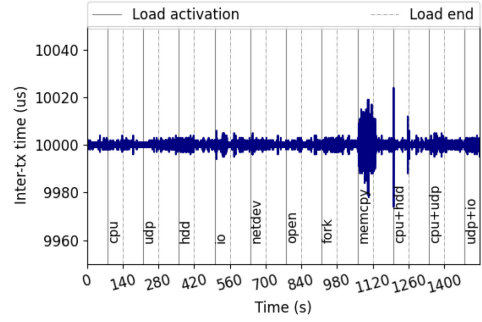


Fig. 9. Mixed workload inter-sending times for network traffic in *Xenomai RTnet* configuration.

Regarding slack times, the second worker node configuration (i.e., *Xenomai no RTnet*) is visibly worse than the others. The reason is that when a Xenomai thread uses a Linux service (e.g., when the thread needs to use a Linux network device driver for support purposes), it will be de-scheduled by Xenomai and scheduled by Linux, even though it is still deemed a Xenomai real-time thread. Evidently, such a scenario heavily impacts scheduling latencies. This is confirmed by the fact that slack times are undermined but activation latencies are still stable. Indeed, repeating the same experiment by avoiding triggering network traffic during the *rt-app* execution against the Xenomai configuration, we obtained slack time depicted in Figure 8.

Concerning the inter-sending time for network packets, Figure 7 shows that, across all the target configurations, the spikes do not always correspond to spikes in the activation latencies and slack times. Hence, different resources do not suffer stress loads in the same way, and they must be isolated and evaluated independently. Further, the inter-sending time for network packets is almost symmetric because a delayed packet implies unbalanced intervals with the packets sent before and after. Finally, Figure 7 shows that inter-sending times for network packets are very stable in the *Xenomai RTnet* configuration. Anyway, by reducing the y-axis limits between $9,950 \mu\text{s}$ and $10,050 \mu\text{s}$ (see Figure 9), we can state that the inter-sending times suffer anyway the co-located stress load, in particular the *memcpy* stressor.

Focusing on the behavior of *rt-app* against the PREEMPT_RT configuration, the effect of stressors on the slack time and activation latency depends on the number of concurrent stressors, as shown in Figure 10. A drop in performance, especially due to the *memcpy* stressor, is evident starting from four concurrent stressors, i.e., when there is one stressor per physical CPU core. This makes sense since the stressor can cause evictions in the first-level cache (L1) of the real-time task when it is scheduled, leaving no room for cached pages.

Summary. Based on the obtained results, the following findings arise:

- Different task-level metrics regarding shared resources can be impacted independently by co-located stress loads. Therefore, the different worker node configurations provide, both at the hardware and software level, a differentiated *level of assurance* regarding the isolation support for each resource (e.g., CPU, I/O, network, memory, etc.).
- All the target configurations suffer from co-located memory-intensive stress load (i.e., *memcpy* stressor). This behavior is due to the lack of mechanisms that address cache isolation (e.g., cache coloring) at the software level. If available, hardware-level mechanisms like Intel CAT [61] and Intel TCC [62] could be used if supported by the container runtime.

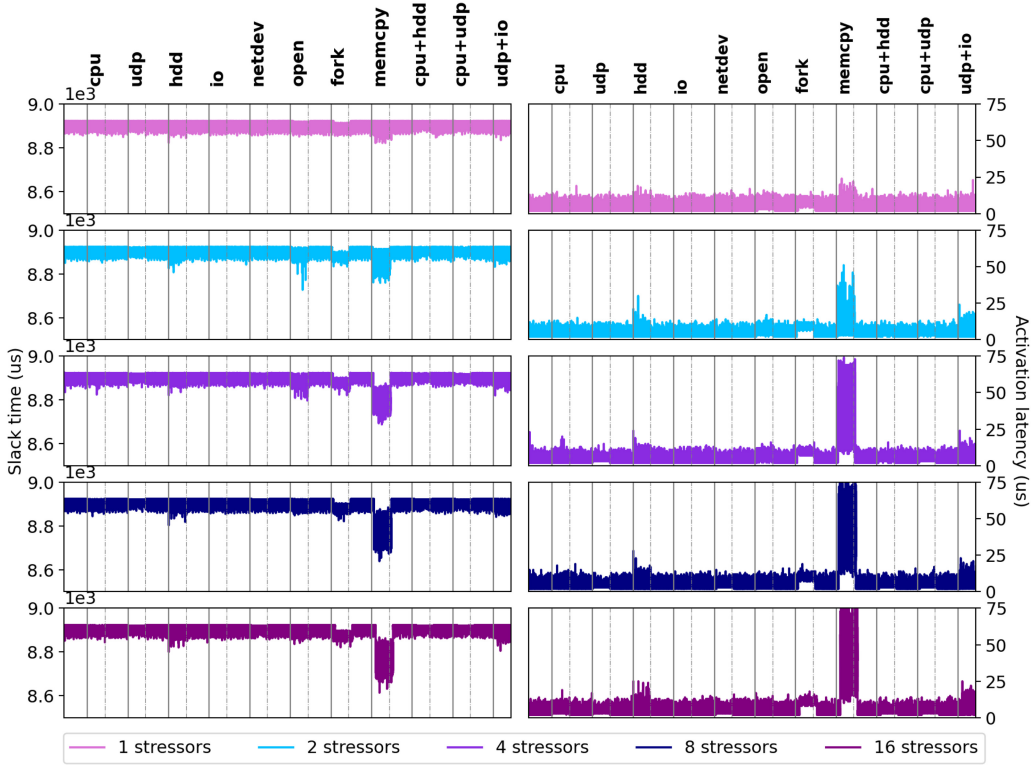


Fig. 10. Mixed workload behavior against the PREEMPT_RT configuration, with increasing load (1, 2, 4, 8, 16 concurrent stressors).

- Concerning slack times, *Xenomai no RTnet* is worse than PREEMPT_RT. This happens because Xenomai threads, after requesting Linux kernel services, are scheduled in the Linux domain, which was not properly configured for real time. Hence, the overall system can benefit from a careful configuration of the Linux kernel besides Xenomai.
- A shared hardware resource can provide different levels of isolation depending on the software stack. For example, inter-sending times have turned out to be more stable than other configurations by adding the real-time networking stack support (i.e., RTnet) to Xenomai, despite using the same network controller. However, even *Xenomai RTnet* still showed little but noticeable increase in the order of approximately 20 μ s.

7.2 Node-level Metrics Selection

Measurements obtained in the previous experiments show that the sensitivity to co-located stress loads may vary between several task-level metrics. Based on this evidence, in this section we show how the workflow presented in Section 6.3 can be used to decide the most useful metrics on which the monitoring system must rely. The selection is made according to the pair (workload, worker node configuration). The monitoring system must take into account the sensitivity of each resource of the worker node to co-located stress loads, in order to monitor more carefully resources with lower assurance. Conversely, it cannot rely on task-level metrics, which are not always available. This kind of monitoring could be implemented by the *resource assurance monitor* (see the last block in Figure 5) and can temporarily allow stress conditions for a strongly isolated resource; conversely,

it must timely notify and react to any hazardous condition for a poorly isolated resource, which is fundamental for task operations. For example, a real-time workload that performs network activity, e.g., sending control signals toward a PLC, could consider foremost delivering network packets by a specified deadline than finishing its activities with a positive slack time. Based on this, that real-time workload could be placed on a worker node that guarantees better isolation for the networking than others. If no worker node with sufficient network isolation level is available, the *resource assurance monitor* could be complemented by further monitoring routines that perform fine-grained checks for issues and reactive network isolation. Therefore, when a workload is running on a worker node, the *resource assurance monitor* must be reconfigured to adapt to incoming workload requirements, being aware of the degree of assurance provided by the node for each of the resources it manages.

Experiment description. We applied the proposed reasoning process against the three worker node configurations analyzed in Section 7.1, i.e., (1) Linux with `rt-cgroups` enabled and `PREEMPT_RT` patch, (2) Xenomai-patched Linux with standard Linux networking stack, and (3) Xenomai-patched Linux with `RTnet` enabled. We used as task-level metrics the activation latency, the slack time, and the inter-sending time for network traffic. The node-level metrics were obtained by periodically sampling processor-related metrics with `mpstat`, I/O-related ones with `iostat`, and the average load with `uptime`. These three Linux applications read data from `/proc/stat`, `/proc/uptime`, `/proc/loadavg`, and `/proc/softirqs` special files. We also collected metrics regarding network traffic, through `vnstat`, and LLC load/store and misses with `perf`, which uses CPU performance counters. The sampling period is set to 3 seconds, to have low overhead on the monitored worker node. Further, we normalized node-level metrics to have unitary *energy* (i.e., the sum of squares) and used 100 samples for the max filter.

Results. Figure 11 shows the samples of the five node-level metrics with the highest scores, obtained across the target worker node configurations. The weights for activation latency, slack time, and inter-sending time for network traffic are respectively 1, 0, and 0 (i.e., activation latency is more important). Figure 12 shows instead the top 5 metrics obtained across the target worker node configurations, with weights for activation latency, slack time, and inter-sending time for network traffic respectively of 0, 0, and 1 (i.e., inter-sending time is more important). The node-level metrics are different for the three configurations: the most important metrics for `PREEMPT_RT` reveal memory-intensive activity; indeed, the *memcpy* is a particularly troublesome stress load for this worker node configuration. Instead, in the *Xenomai RTnet* configuration the most important node-level metrics are related to timer interrupts (i.e., *LOC/s* metric) and interrupt-related (i.e., *TIMER/s*, *intr/s* metrics) workloads. It is important to note that, even if a single node-level metric is not able to selectively identify a guilty stress type, a linear combination of them allows this. For example, the *TIMER/s* metric increase, for the Xenomai configurations (both with and without `RTnet`), is noticeable across almost all the stress loads. However, if we put together such information with the *SCHED/s* metric, we can easily identify *fork* as intensive workload. Moreover, a metric is not very helpful alone, like for *usr%* in the `PREEMPT_RT` configuration, which is triggered by both CPU-bound and memory-intensive stressors, but analyzing the top 5 node-level metrics together, it is possible to identify the nature of the stress load. By switching to the inter-sending time for network traffic as the most important task-level metric (see Figure 12), the ranking for node-level metrics is clearly different. For example, for the `PREEMPT_RT` configuration, the memory-related metrics are not the most important anymore as for the previous scenario. Indeed, the *memcpy* stressor has a similar effect compared to the other high-impact stress loads (e.g., *open*, *cpu+hdd*, *cpu+udp*) for the inter-sending time task-level metric.

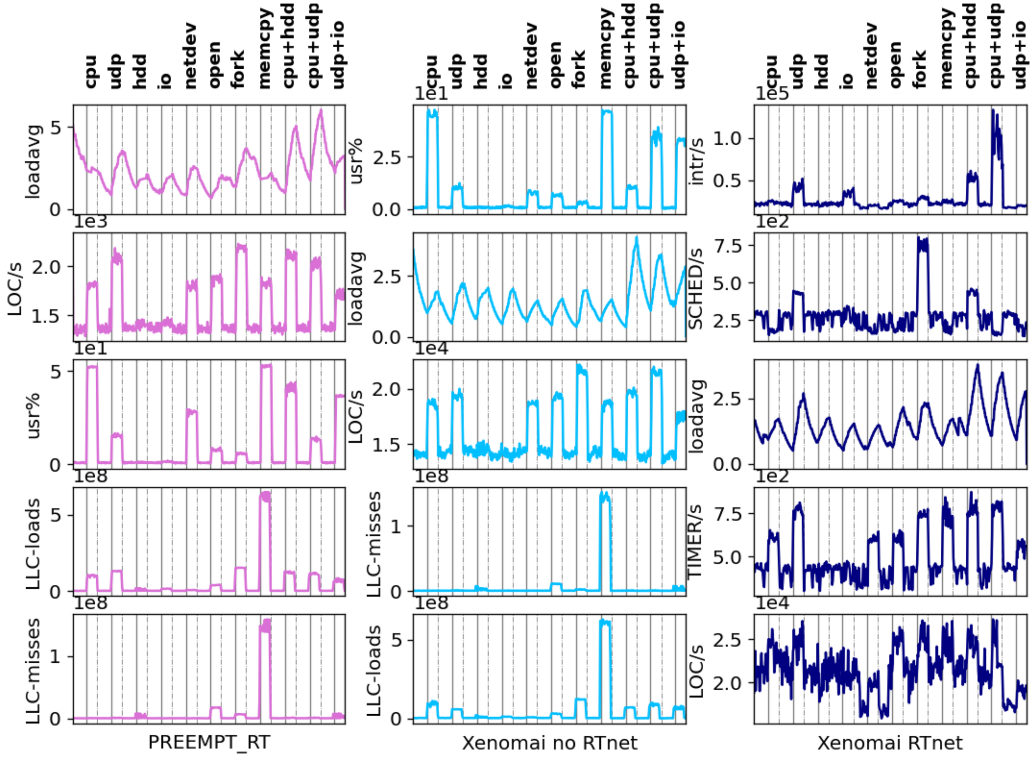


Fig. 11. Monitoring traces with activation latency as the most important task-level metric. The metrics are ordered by increasing score from the top to the bottom.

Summary. Summing up, we can derive the following findings:

- The node-level metrics that the resource assurance monitor must sample are prioritized differently depending on the weights given by a job for the several task-level metrics.
- The node-level metrics to be considered can differ based on the underlying system configuration, which can suffer more a stress load rather than another, and they are polarized toward the identification of the most hazardous conditions.
- The node-level metrics must be considered jointly to infer the cause of the stress load, since each of them taken alone could not be helpful to understand the actual state of the system.

7.3 Scheduling

In this section, we compare our proposal for criticality-aware placement against the default scheduler provided by Kubernetes (in the following, *vanilla* scheduler). To this aim, we consider a setup similar to what is described at the beginning of Section 7. Specifically, the Kubernetes cluster includes three worker nodes configured as follows:

- *Worker Node 1 (WN1)*: PREEMPT_RT patched Linux kernel with `rt-cgroups`
- *Worker Node 1 (WN2)*: Xenomai-patched Linux with standard Linux networking stack
- *Worker Node 1 (WN3)*: *vanilla* Linux kernel

We show how our proposal gives valuable hints to the scheduler to correctly place real-time *Pods*. Next, we compare the schedulability of Kubernetes and our prototype.

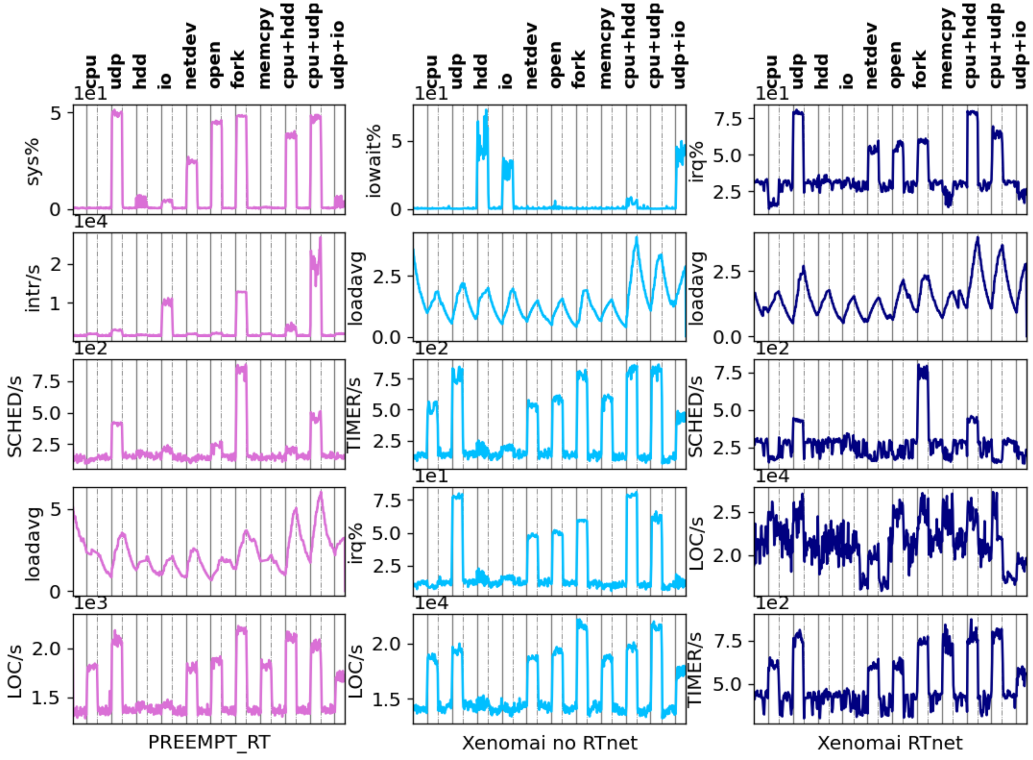


Fig. 12. Monitoring traces with inter-sending time for network traffic as the most important task-level metric. The metrics are ordered by increasing score from the top to the bottom.

It is worth noticing that our plugin is not essential to give placement hints to the scheduler regarding the real-time assurance of the nodes; indeed, we use labels and annotations without modifying Kubernetes, in the same way as a common user could do. However, manually setting the labels would not take advantage of the monitoring system, with no effect on the *aggregation policy*.

7.3.1 Criticality-aware Placement.

Experiment description. We assume a real-time Pod running the *rt-app*, like in previous sections. We simulate a placement decision made by Kubernetes with no hints of such a real-time Pod and retrieve the task-level metrics over time. Further, we compare gathered task-level metrics against the same placement decision made by our criticality-aware scheduler (see Section 6.1) in terms of *normalized response time*.

Results. Figure 13 shows the normalized response time for the aggregated metrics. The Kubernetes *vanilla* scheduler places the Pod on WN1 and WN3; instead, the criticality-aware placement engine in K4.0s places the Pod on WN1 and WN2. Clearly, the placement of the real-time Pod on WN3 leads to several deadline misses (i.e., the normalized response time exceeds 1) compared to our proposal, in which there is no deadline misses for the real-time Pod.

7.3.2 Schedulability.

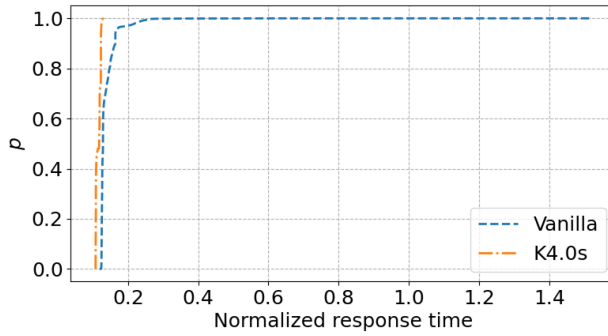


Fig. 13. The cumulative distribution functions of the normalized response time between Kubernetes *vanilla* scheduler and the proposed K4.0s.

Experiment description. Differently from the plugin for the assurance level, plugins for network and real-time schedulability cannot be replaced by user actions unless manually computing the schedulability tests and forcing a Pod to a designed node. In this second experiment, we compare the schedulable sets of Pods between vanilla Kubernetes and our prototype. We simulated the placement of Pods in a cluster made up of five nodes with no real-time guarantees, two nodes with medium real-time isolation guarantees that allow a *budget/period* scheduling interface for containers (with the schedulability test used in [24]), and one node with high real-time isolation guarantees that is characterized by a hierarchical deferrable server scheduling. The set of Pods to be scheduled is composed of (1) 30 best-effort Pods with random CPU requirements between 8% and 25%; (2) 30 real-time Pods with medium isolation requirements, with random real-time CPU requirements between 8% and 25%; and (3) a random number between 6 and 12 of real-time Pods with high isolation requirements, created with the algorithm presented in [63] such that their total utilization is 180%.

Results. In our simulation, Kubernetes considers all 200 sets of Pods as schedulable, while our proposal schedules only 108 of the 200 sets. The remaining sets are not schedulable since not enough resources can be guaranteed, in the worst case, to complete the execution on time. In particular, 61 sets were rejected by the hierarchical deferrable server schedulability test, and 45 were rejected by the EDF utilization bound. Hence, even if Kubernetes would schedule all the Pods, their execution will lead to hazardous latencies and deadline misses.

8 CONCLUSION AND FUTURE WORKS

We proposed an extension of the system model underlying orchestration solutions in order to enable the use of orchestrators in I4.0 scenarios with mixed-criticality and timeliness requirements. The proposed model includes novel abstractions that consider the assurance of the resources of the nodes, the criticality of workloads, networking topologies, and a suitable real-time scheduling interface for the jobs. A monitoring and orchestration architecture based on the proposed model has been presented, which extends the reference architecture of orchestration systems introducing new components that deal with proposed abstractions. We also sketched a Kubernetes-based implementation of the proposal.

Through an experimental campaign, we demonstrated that each resource of a node is characterized by a real-time assurance independent of the others. Further, we showed that the proposed assurance level can drive the selection of proper monitoring metrics by proposing a workflow for selecting monitoring node-level metrics considering the characteristics of both worker node and

workload. In this way, the resource assurance monitor on the worker node can be reconfigured to adapt to the requirements of a workload, providing differentiated guarantees to real-time critical workloads. Finally, concerning real-time scenarios, the proposed criticality-aware scheduler allows properly placing workloads according to the requested assurance against unaware choices made by the current version of Kubernetes.

Our proposal is one of the first steps toward the support of container orchestration for real-time scenarios. Despite the research efforts into the study of end-to-end orchestration availability, there are still research gaps in the understanding of the predictability bottlenecks for downtimes, in order to derive upper bounds for the Time-to-Repair. For example, the migration time needed by Kubernetes to replace a failed Pod consists of several contributions, which need to be lowered as much as possible and designed to be predictable. Similarly, even if Kubernetes offers a high-availability configuration for the control plane, limits, consequences on timings, and failure models have not been investigated properly yet. Therefore, we plan to perform systematic experimentation to reveal precise points to make adjustments (e.g., within the Kubernetes control plane), for example, on what scale the orchestrator can detect and mitigate interferences between real-time and non-real-time applications.

REFERENCES

- [1] Heiner Lasi, Peter Fettke, Hans-Georg Kemper, Thomas Feld, and Michael Hoffmann. 2014. Industry 4.0. *Springer Business & Information Systems Engineering (BISE)* 6 (2014), 239–242.
- [2] A.-W. Colombo, S. Karnouskos, and J.-M. Mendes. 2010. Factory of the future: A service-oriented system of modular, dynamic reconfigurable and collaborative systems. *Springer Artificial Intelligence Techniques for Networked Manufacturing Enterprises Management* (2010), 459–481.
- [3] Bjarne Johansson, Mats Rågberger, Thomas Nolte, and Alessandro V. Papadopoulos. 2022. Kubernetes orchestration of high availability distributed control systems. In *IEEE International Conference on Industrial Technology (ICIT'22)*. IEEE, 1–8.
- [4] Jacob Mellado and Felipe Núñez. 2022. Design of an IoT-PLC: A containerized programmable logical controller for the industry 4.0. *Elsevier Journal of Industrial Information Integration* 25 (2022), 100250.
- [5] Emiliano Sisinni, Abusayeed Saifullah, Song Han, Ulf Jennehag, and Mikael Gidlund. 2018. Industrial Internet of Things: Challenges, opportunities, and directions. *IEEE Transactions on Industrial Informatics (TII)* 14, 11 (2018), 4724–4734.
- [6] Andrea Borghesi, Giuseppe Di Modica, Paolo Bellavista, Varun Gowtham, Alexander Willner, Daniel Nehls, Florian Kintzler, Stephan Cejka, Simone Rossi Tisbeni, Alessandro Costantini, et al. 2021. Iotwins: Design and implementation of a platform for the management of digital twins in industrial scenarios. In *Proceedings of 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid'21)*. IEEE, 625–633.
- [7] Spyridon V. Gogouvis, Harald Mueller, Sreenath Premnadh, Andreas Seitz, and Bernd Bruegge. 2020. Seamless computing in industrial systems using container orchestration. *Elsevier Future Generation Computer Systems* 109 (2020), 678–688.
- [8] Ranesh Kumar Naha, Saurabh Garg, Dimitrios Georgakopoulos, Prem Prakash Jayaraman, Longxiang Gao, Yong Xiang, and Rajiv Ranjan. 2018. Fog computing: Survey of trends, architectures, requirements, and research directions. *IEEE Access* 6 (2018), 47980–48009.
- [9] Alessandro Cilaro, Marcello Cinque, Luigi De Simone, and Nicola Mazzocca. 2022. Virtualization over multiprocessor systems-on-chip: An enabling paradigm for the industrial Internet of Things. *IEEE Computer* 55, 10 (2022), 35–47.
- [10] Alan Burns and Robert Ian Davis. 2022. Mixed criticality systems-a review. Department of Computer Science, University of York, Tech. Rep (2022).
- [11] Gernot Heiser. 2011. Virtualizing embedded systems: Why bother? In *Proceedings of the 48th Design Automation Conference*. 901–905.
- [12] BlackBerry Limited. 2021. Are Hypervisors the Answer to the Coming Silicon Shortages? (White Paper). https://blackberry.qnx.com/content/dam/blackberry-com/Documents/pdf/BlackBerry_QNX_Hypervisor_WhitePaper_22April2021_FINAL.pdf.
- [13] Marcello Cinque, Domenico Cotroneo, Luigi De Simone, and Stefano Rosiello. 2022. Virtualizing mixed-criticality systems: A survey on industrial trends and issues. *Elsevier Future Generation Computer Systems (FGCS)* 129 (2022), 315–330.

- [14] Roberto Morabito, Vittorio Cozzolino, Aaron Yi Ding, Nicklas Beijar, and Jorg Ott. 2018. Consolidate IoT edge computing with lightweight virtualization. *IEEE Network* 32, 1 (2018), 102–111.
- [15] Václav Struhár, Moris Behnam, Mohammad Ashjaei, and Alessandro V. Papadopoulos. 2020. Real-time containers: A survey. In *Proceedings of 2nd Workshop on Fog Computing and the IoT (Fog-IoT'20)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik.
- [16] Marcello Cinque, Raffaele Della Corte, Antonio Eliso, and Antonio Pecchia. 2019. Rt-cases: Container-based virtualization for temporally separated mixed-criticality task sets. In *Proceedings of 31st Euromicro Conference on Real-time Systems (ECRTS'19)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.
- [17] Allison Randal. 2020. The ideal versus the real: Revisiting the history of virtual machines and containers. *ACM Computing Surveys (CSUR)* 53, 1 (2020), 1–31.
- [18] Docker Inc. 2022. Swarm Mode Overview. Retrieved June 22, 2023, from <https://docs.docker.com/engine/swarm/>.
- [19] The Linux Foundation. 2022. Kubernetes Home Page. Retrieved June 22, 2023, from <https://kubernetes.io/>.
- [20] Apache Software Foundation. 2022. Apache Mesos Home Page. Retrieved June 22, 2023, from <https://mesos.apache.org/>.
- [21] Asif Khan. 2017. Key characteristics of a container orchestration platform to enable a modern application. *IEEE Cloud Computing* 4, 5 (2017), 42–48.
- [22] Maria A. Rodriguez and Rajkumar Buyya. 2019. Container-based cluster orchestration systems: A taxonomy and future directions. *Wiley Software: Practice and Experience* 49, 5 (2019), 698–719.
- [23] Carmen Carrión. 2022. Kubernetes scheduling: Taxonomy, ongoing issues and challenges. *Comput. Surveys* 55, 7 (2022), 1–37.
- [24] Stefano Fiori, Luca Abeni, and Tommaso Cucinotta. 2022. RT-Kubernetes—containerized real-time cloud computing. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*. 36–39.
- [25] László Toka. 2021. Ultra-reliable and low-latency computing in the edge with Kubernetes. *Journal of Grid Computing* 19, 3 (2021), 31.
- [26] Václav Struhár, Silviu S. Craciunas, Mohammad Ashjaei, Moris Behnam, and Alessandro V. Papadopoulos. 2021. RE-ACT: Enabling real-time container orchestration. In *Proceedings of 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA'21)*. IEEE, 1–8.
- [27] Raphael Eidenbenz, Yvonne-Anne Pignolet, and Alain Ryser. 2020. Latency-aware industrial fog application orchestration with Kubernetes. In *Proceedings of 5th IEEE International Conference on Fog and Mobile Edge Computing (FMEC'20)*. IEEE, 164–171.
- [28] Gabriele Monaco, Gautam Gala, and Gerhard Fohler. 2023. Extensions for shared resource orchestration in Kubernetes to support RT-cloud containers. *arXiv preprint arXiv:2301.07479* (2023).
- [29] Sebastian Böhm and Guido Wirtz. 2022. Towards orchestration of cloud-edge architectures with Kubernetes. In *Springer Science and Technologies for Smart Cities*. Springer, 207–230.
- [30] Eddy Truyen, Dimitri Van Landuyt, Davy Preuveneers, Bert Lagaisse, and Wouter Joosen. 2019. A comprehensive feature comparison study of open-source container orchestration frameworks. *MDPI Applied Sciences* 9, 5 (2019), 931.
- [31] Xilinx. 2022. RunX. Retrieved June 22, 2023, from <https://github.com/Xilinx/runx>.
- [32] Katacontainers. 2017. Home Page of Katacontainers. Retrieved June 22, 2023, from <https://katacontainers.io/>.
- [33] Marco Barletta, Marcello Cinque, Luigi De Simone, and Raffaele Della Corte. 2022. Introducing k4. 0s: A model for mixed-criticality container orchestration in industry 4.0. In *2022 IEEE Intl. Conf. on Dependable, Autonomic and Secure Computing, Intl. Conf. on Pervasive Intelligence and Computing, Intl. Conf. on Cloud and Big Data Computing, Intl. Conf. on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech'22)*. IEEE, 1–6.
- [34] Kubernetes. 2022. Kubernetes Metrics Server. Retrieved June 22, 2023, from <https://github.com/kubernetes-sigs/metrics-server>.
- [35] Cloud Native Computing Foundation. 2022. Retrieved June 22, 2023, from Prometheus. <https://prometheus.io/>.
- [36] Influx Data. 2022. Telegraf. Retrieved June 22, 2023, from <https://docs.influxdata.com/telegraf/>.
- [37] VictoriaMetrics. 2022. VictoriaMetrics. Retrieved June 22, 2023, from <https://github.com/VictoriaMetrics/VictoriaMetrics>.
- [38] Luca Abeni, Alessio Balsini, and Tommaso Cucinotta. 2019. Container-based real-time scheduling in the Linux kernel. *ACM Special Interest Group on Embedded Systems (SIGBED) Review* 16, 3 (2019), 33–38.
- [39] Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, Riccardo Mancini, and Carlo Vitucci. 2021. Strong temporal isolation among containers in OpenStack for NFV services. *IEEE Transactions on Cloud Computing (TCC)* 11, 1 (2021), 763–778.
- [40] Sisu Xi, Chong Li, Chenyang Lu, Christopher D. Gill, Meng Xu, Linh T. X. Phan, Insup Lee, and Oleg Sokolsky. 2015. Rt-open stack: Cpu resource management for real-time cloud computing. In *Proceedings of 8th IEEE International Conference on Cloud Computing*. IEEE.

- [41] Mark Szalay, Peter Matray, and Laszlo Toka. 2022. Real-time FaaS: Towards a latency bounded serverless cloud. *IEEE Transactions on Cloud Computing (TCC)* 11, 2 (2022), 1636–1650.
- [42] Hylson V. Netto, Lau Cheuk Lung, Miguel Correia, Aldelir Fernando Luiz, and Luciana Moreira Sá de Souza. 2017. State machine replication in containers managed by Kubernetes. *Elsevier Journal of Systems Architecture* (2017), 134–135.
- [43] Marco Barletta, Marcello Cinque, Luigi De Simone, Raffaele Della Corte, Giorgio Farina, and Daniele Ottaviano. 2022. RunPHI: Enabling mixed-criticality containers via partitioning hypervisors in industry 4.0. In *2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW'22)*. 134–135. DOI : <http://dx.doi.org/10.1109/ISSREW55968.2022.00058>
- [44] Wei-Tek Tsai, Qihong Shao, Xin Sun, and Jay Elston. 2010. Real-time service-oriented cloud computing. In *World Congress on Services*. IEEE.
- [45] W. T. Tsai, Yann-Hang Lee, Zhibin Cao, Yinong Chen, and Bingnan Xiao. 2006. RTSOA: Real-time service-oriented architecture. In *Proceedings of 2nd IEEE International Symposium on Service-oriented System Engineering*. IEEE, 49–56.
- [46] Tommaso Cucinotta, Antonio Mancina, Gaetano F. Anastasi, Giuseppe Lipari, Leonardo Mangeruca, Roberto Chec-cozzo, and Fulvio Rusinà. 2009. A real-time service-oriented architecture for industrial automation. *IEEE Transactions on Industrial Informatics* 5, 3 (2009), 267–277.
- [47] Hendrik Bohn, Andreas Bobek, and Frank Golasowski. 2006. SIRENA-service infrastructure for real-time embedded networked devices: A service oriented framework for different domains. In *Proceedings of International Conference on Networking, International Conference on Systems and International Conference on Mobile Communications and Learning Technologies (ICNICONSMCL'06)*. IEEE, 43–43.
- [48] Carolyn McGregor and J. Mikael Eklund. 2010. Next generation remote critical care through service-oriented architectures: Challenges and opportunities. *Springer Service Oriented Computing and Applications* 53 (2010).
- [49] Isam Mashhour Al Jawarneh, Paolo Bellavista, Filippo Bosi, Luca Foschini, Giuseppe Martuscelli, Rebecca Montanari, and Amedeo Palopoli. 2019. Container orchestration engines: A thorough functional and performance comparison. In *IEEE International Conference on Communications*. 1–6.
- [50] Real-Time Group Scheduling. 2022. Real-Time Group Scheduling. Retrieved June 22, 2023, from <https://www.kernel.org/doc/Documentation/scheduler/sched-rt-group.txt>.
- [51] Marco Barletta, Marcello Cinque, Luigi De Simone, and Raffaele Della Corte. 2022. Achieving isolation in mixed-criticality industrial edge systems with real-time containers. In *Proceedings of 34th Euromicro Conference on Real-Time Systems (ECRTS'22)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik.
- [52] Algirdas Avizienis, J.-C. Laprie, Brian Randell, and Carl Landwehr. 2004. Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing (TDSC)* 1, 1 (2004), 11–33.
- [53] Sinem Coleri Ergen and Pravin Varaiya. 2010. TDMA scheduling algorithms for wireless sensor networks. *Springer Wireless Networks* 16 (2010), 985–997.
- [54] Steve Vestal. 2007. Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance. In *Proceedings of 28th IEEE International Real-time Systems Symposium (RTSS'07)*. IEEE, 239–243.
- [55] K4.0s. 2022. k4.0s Gitlab Repository. <https://dessert.unina.it:8088/marcobarlo/k4.0s>.
- [56] Heiko Koziolk and Nafise Eskandani. 2023. Lightweight Kubernetes distributions: A performance comparison of MicroK8s, k3s, k0s, and microshift. In *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering*. 17–29.
- [57] Cloud Native Computing Foundation. 2022. Kubernetes Certified Distributions. Retrieved June 22, 2023, from <https://www.cncf.io/certification/software-conformance/>.
- [58] Jan Kiszka and Bernardo Wagner. 2005. RTnet-a flexible hard real-time networking framework. In *Proceedings of IEEE Conference on Emerging Technologies and Factory Automation*. IEEE.
- [59] Guowang Miao, Jens Zander, Ki Won Sung, and Slimane Ben Slimane. 2016. *Fundamentals of Mobile Data Networks*. Cambridge University Press.
- [60] Xiao Zhang, Sandhya Dwarkadas, and Kai Shen. 2009. Towards practical page coloring-based multicore cache management. In *Proceedings of ACM European Conference on Computer Systems*.
- [61] Intel. Cache Allocation Technology in Intel Xeon Processor. Retrieved June 22, 2023, from <https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-cache-allocation-technology.html>.
- [62] Intel. Overview of Intel® Time Coordinated Computing (TCC) Tools. Retrieved June 22, 2023, from <https://www.intel.com/content/www/us/en/developer/articles/technical/real-time-systems-measurement-library.html>.
- [63] Paul Emberson, Roger Stafford, and Robert I. Davis. 2010. Techniques for the synthesis of multiprocessor tasksets. In *Proceedings of International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems (WATERS'10)*.

Received 10 November 2022; revised 16 March 2023; accepted 4 June 2023